Chain Reaction: Nexus Protocol — Design, Implementation, and Empirical Evaluation of a Weighted-Heuristic AI Opponent for a Browser-Based Multiplayer Strategy Game

Md. Tahmidur Rahman Nafees (2022454642) · Sakib Rahman Rohan (2011350042)
tahmidur.nafees@northsouth.edu · rahman.rohan@northsouth.edu
Department of Electrical and Computer Engineering, North South University
Submitted to: Mohammad Shifat-E-Rabbi <rabbi.mohammad@northsouth.edu>
Final Report · August 27, 2026 · Repository: github.com/tahmidnafees619/ChainReaction

***Abstract*—**We present *Chain Reaction: Nexus Protocol*, a browser-based implementation of the classic orb/critical-mass capture-strategy game, supporting 2–10 players in any combination of human and computer-controlled participants on a configurable grid. The system is built as a dependency-free client-side application: a headless, event-driven rules engine; a single-ply, simulate-and-score heuristic evaluator that selects moves for the computer opponent across three difficulty tiers; and a Canvas-2D-rendered frontend with procedurally synthesized audio. We formalize the game's cascade-resolution algorithm and the AI's scoring function, including its six evaluation features and per-difficulty weight vectors, and support the design with a worked numerical example showing the same board position ranked differently by two difficulty tiers. We evaluate the system along two axes: correctness/robustness, via 126 automated test assertions spanning a Node.js unit-test tier and a browser end-to-end tier, the latter of which we show was necessary to catch three real defects invisible to the former; and AI strength, via a 300-game self-play tournament across all difficulty pairings, showing a monotonic and statistically clear win-rate gradient consistent with the intended difficulty ordering. We conclude with a discussion of design trade-offs, threats to validity, and directions for future work.

***Index Terms*—**game artificial intelligence, heuristic evaluation function, web application testing, HTML5 Canvas, procedural audio, software engineering case study

I. Introduction

Chain Reaction is a turn-based capture-strategy game played on a rectangular grid, in which players place indivisible "orbs" into cells they own or that are empty. Each cell has a position-dependent *critical mass*: once its orb count reaches that threshold, the cell explodes, emptying itself and firing one orb into each orthogonally adjacent cell, converting the ownership of every such neighbor to the exploding player. A single placement can therefore trigger a cascading sequence of further explosions — a chain reaction — that sweeps across a large fraction of the board in one turn. A player is eliminated once every participant has moved at least once and that player controls zero orbs; the last player remaining wins. The game's appeal, and its difficulty as a target for both human strategy and computer opponents, follows directly from this mechanic: a single move's consequences are highly non-local and can be very large relative to board size, which makes naive move evaluation unreliable and rewards actually simulating a candidate move's outcome rather than judging it by superficial position alone.

This project, *Chain Reaction: Nexus Protocol*, implements the full ruleset as a self-contained, dependency-free web application, together with a non-trivial computer opponent and a cyberpunk-themed visual and audio presentation layer. Three properties distinguish the engineering approach taken here from a typical class-project implementation of this game:

1. The rules engine is **headless** — it has no dependency on the DOM, Canvas, or any rendering technology, and communicates exclusively through a documented event contract. This makes the rules independently testable under a plain Node.js process and, in principle, reusable behind an entirely different frontend.
2. The computer opponent is a **single-ply simulate-and-score evaluator**: rather than a hand-written decision tree or a deep minimax search, it fully resolves the cascade each candidate move would trigger on a disposable copy of the board, extracts a small set of numeric features from the resulting position, and combines them via a difficulty-specific weighted sum. Section IV formalizes this precisely and demonstrates, with a worked example, that difficulty level is entirely a function of the weight vector and a randomness parameter — not of different code paths.
3. The project is verified by **two independent test tiers** — a DOM-free unit tier and a real-browser end-to-end tier — and we report, as a methodological finding in its own right, that several genuine defects (a permanently frozen game state, a completely unclickable UI control, and a race condition triggering an uncaught exception) were only discoverable by the second tier. This is presented as an empirical case for automated end-to-end verification of client-side applications generally, not only for this project.

The remainder of this report is organized as follows. Section II situates the project relative to existing implementations of the same game and to general game-AI evaluation-

function literature. Section III describes the system architecture and formalizes the core game rules. Section IV gives the complete AI methodology. Section V describes the implementation. Section VI reports our experimental evaluation, covering both correctness testing and AI strength. Section VII discusses the results and the project's limitations. Section VIII concludes.

II. Related Work

The orb/critical-mass capture mechanic implemented here is not original to this project; grid capture games built on this mechanic have been published under the name "Chain Reaction" across numerous independent mobile and web implementations, including a well-documented Android release by Buddy-Matt Entertainment [1]. Rule variants differ primarily in the exact capacity assigned to corner, edge, and interior cells; this project uses capacity equal to a cell's orthogonal-neighbor count (2/3/4 for corner/edge/interior), whereas at least one open-source implementation we examined uses a one-lower convention (1/2/3) [2]. This project's engine (Section III-B) is written to derive capacity from neighbor count as a general rule rather than hard-coding the four possible values, which makes such rule variants a configuration choice rather than a code change, though only the 2/3/4 convention is exposed in the current build.

That same open-source implementation [2] is, to our knowledge, the most directly comparable prior system: a two-player version of the same game with a computer opponent driven by minimax search with alpha-beta pruning. This represents the opposite point in the design space from the approach taken here — a deeper but more expensive search over a coarser position evaluation, versus this project's single-ply but feature-rich evaluation (Section IV). We are not aware of a published head-to-head comparison between the two approaches on this specific game, and we note it as a natural direction for future work (Section VIII).

More broadly, the idea of replacing exhaustive search with a fast, hand-engineered position evaluation function is one of the oldest ideas in game AI, dating at least to Shannon's foundational proposal for a chess-playing program that scores positions via a weighted combination of material and positional features rather than searching to the end of the game [3]. The AI described in Section IV follows this same design philosophy — a linear combination of hand-chosen features, Section IV-D — adapted to a domain where, unlike chess, a single move's effect on the evaluated features is only knowable by actually resolving a cascade rather than by inspecting the move in isolation, which is why the evaluator simulates each candidate move (Section IV-B) rather than scoring board deltas analytically.

III. System Architecture and Game Rules

A. Architectural Overview

The system is partitioned into three modules communicating through a narrow, explicit interface, shown in Fig. 1. `ChainReactionEngine.js` owns all game state (a two-dimensional array of cells, turn order, active-player set) and exposes exactly one mutating entry point, `handlePlayerClick(row, col)`, plus a subscription method, `on(event, callback)`, through which it emits seven distinct event types (state changes, per-cell explosions, turn changes, eliminations, game-over, invalid-move errors, and a cascade-safety-cap notice) documented as a formal contract in the module's header comment. `ChainReactionAI.js` consumes a read-only snapshot of engine state and returns a move; it never mutates the live engine (Section IV). `main.html` is the sole consumer of both: it renders the board to a 2-D canvas, synthesizes audio, manages menu and in-game screens, and is the only module aware that a DOM exists at all. No module other than `main.html` references `window` or `document`.

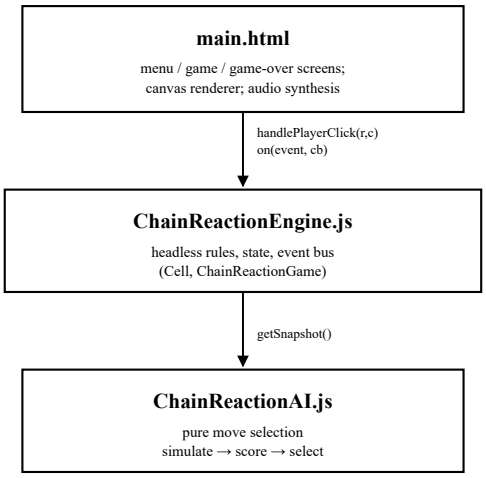

Fig. 1. Module decomposition and control flow. Arrows indicate the direction of a call; the AI module never mutates engine state.

B. Critical Mass and Cascade Resolution

Let a board have R rows and C columns. The critical mass (capacity) of the cell at row r, column c is defined as its orthogonal-neighbor count:

$$cap(r,c) = 4 - [r \in \{0, R-1\}] - [c \in \{0, C-1\}] \quad (1)$$

using Iverson-bracket notation, which yields 2 for a corner cell, 3 for a non-corner edge cell, and 4 for an interior cell. A cell is *critical* when its orb count meets or exceeds its capacity. Rather than resolving explosions recursively as they are triggered, the engine resolves them in discrete simultaneous *waves*: on each iteration, every currently-critical cell is collected, all such cells explode together (each losing exactly $cap(r,c)$ orbs and distributing one orb to each orthogonal neighbor, with ownership of every recipient cell transferring to the player who caused the wave), and the board is rescanned; this repeats until no cell is critical. This wave-based resolution — rather than a depth-first recursive one — was chosen because it matches the simultaneous character of the physical game and gives a natural, uniform per-wave point at which a frontend can animate and pace the cascade.

Because every exploding cell loses exactly its capacity in orbs and distributes exactly that many orbs to its neighbors, the total orb count on the board is invariant across a wave; a cascade can only terminate because the board eventually reaches a configuration with no critical cell, not because orbs are lost. This is not incidental to this particular game: a vertex holding at least as many tokens as it has neighbors, toppling by sending one token to each neighbor and repeating until no vertex is unstable, is precisely the definition of a *chip-firing game* on a graph, studied formally in combinatorics and, under a physical framing, as the abelian sandpile model of self-organized criticality [9]. That literature's central caution — that toppling sequences on adversarial or highly symmetric configurations are not guaranteed to terminate quickly — is exactly why this project's resolver additionally enforces a hard ceiling on the number of waves a single cascade may run (Section VI-A discusses this as a discovered robustness requirement, not merely a theoretical one), after which any cells still critical are simply picked up again the next time a wave is resolved. A player already active before the current move is eliminated once every player has moved at least once and that player's total orb count reaches zero; the game ends, and the sole player with remaining orbs is declared the winner, once exactly one player controls any orbs at all.

IV. Artificial Intelligence Methodology

A. Design Rationale

Two broad strategies exist for automated move selection in this domain: search-based methods that look ahead over multiple plies and reply sequences (e.g., minimax, as used in the comparable system discussed in Section II [2]), and evaluation-based methods that judge each candidate move by a single, richer measurement of its immediate consequence. This project adopts the second approach. The decisive practical consideration is that a single move's effect on the board is highly non-local — a placement that looks unremarkable can trigger a cascade that recolors a large fraction of the grid — so the *branching factor* that a search would need to explore at useful depth grows quickly with board size, while the *value* of looking further ahead is dominated by how accurately the immediate consequence of a move is scored in the first place. We therefore invested the AI's complexity budget into a feature-rich, simulation-grounded single-ply evaluator rather than a shallow search over a coarser evaluation, subject to the constraint that move selection must complete well within the interface's artificial "thinking" delay (600 ms) on boards up to 12×12 — a constraint we verify empirically in Section VI-C.

B. Move Simulation

Given the current board state and a player identifier p, the set of legal moves is

$$M = \{ (r,c) : owner(r,c) = \emptyset \vee owner(r,c) = p \} \tag{2}$$

For every candidate move $m \in M$, the evaluator clones the board into a disposable, plain-data copy, places one orb for p at m, and resolves the resulting cascade to a fixed point using the identical wave-based rule given in Section III-B — independently re-implemented over the clone so that the evaluator can never mutate the live game — capped at 200 simulated waves purely to bound per-move evaluation cost. This produces a resulting board $B(m)$ and a wave count $W(m)$.

C. Position Evaluation

Six features are extracted from $B(m)$, summarized in Table I. Let $orbs_p$ denote total orbs owned by player p on $B(m)$, and $orbs_{\neg p}$ the combined total for every other player.

$$orbDelta(m) = orbs_p - orbs_{\neg p} \tag{3}$$

Table I
AI Evaluation Features (computed on B(m))

Feature	Definition	Rationale
orbDelta	Eq. (3): net orb material	overall board control
cellControl	count of cells owned by p	territory, not just orb count
corner	of those, count at a corner	corners cost only 2 orbs to hold
threat	opponent cells ≥ cap−1 adjacent to p	exposure to a counter-chain
eliminated	active opponents left with 0 orbs	knockout progress
isWin	exactly one owner remains, and it is p	terminal move detection

The *threat* feature specifically counts opponent-owned cells that are one orb away from exploding *and* orthogonally adjacent to at least one cell owned by p — i.e., positions from which an opponent could flip the evaluating player's territory on their very next turn. This is the only feature that looks past the immediately resulting position toward near-term risk, and, as Section IV-E shows, is also the feature whose weight varies most sharply across difficulty levels.

D. Scoring Function

The six features are combined into a scalar score by a weighted linear sum, with an overriding bonus for a move that ends the game outright:

$$\begin{aligned} score(m) = {} & w_o \cdot orbDelta + w_c \cdot cellControl + w_k \cdot corner \\ & - w_t \cdot threat + w_e \cdot eliminated + w_h \cdot W(m) + \\ & 10^5 \cdot [isWin(m)] \end{aligned} \tag{4}$$

The weight vector $(w_o, w_c, w_k, w_t, w_e, w_h)$ is the *entire* mechanism by which the three difficulty tiers differ — there is no difficulty-conditional branching anywhere else in the move-selection code. Table II gives the values used in the shipped system.

TABLE II
Per-Difficulty Weights and Selection Parameters

Parameter	Easy	Medium	Hard
w_o (orbDelta)	1.0	1.4	1.8
w_c (cellControl)	0.4	0.8	1.1
w_k (corner)	0.5	0.9	1.1
w_t (threat)	0.0	0.6	1.4
w_e (eliminated)	3.0	6.0	9.0
w_h (chain length)	0.1	0.3	0.5
ρ (randomness)	0.75	0.25	0.05
τ (pool fraction)	0.6	0.3	0.08

E. Move Selection

If any candidate move satisfies $isWin(m)$, it is returned immediately regardless of difficulty, so that no difficulty tier ever fails to close out an available win. Otherwise, all moves are ranked by score, and a random draw $u \sim U(0,1)$ determines the selection rule: if $u < \rho$, a move is drawn uniformly from the top $k = max(1, round(\tau \cdot |M|))$ ranked moves; otherwise, the single highest-scoring move is taken (ties broken uniformly at random). Difficulty is therefore purely a matter of *how much the ranking is trusted*: at $\rho = 0.05$, Hard takes its top-ranked move 95% of the time; at $\rho = 0.75$, Easy substitutes a wide random pool three turns in four, which is what makes it a genuinely weaker and more exploitable opponent rather than the same evaluator playing more slowly.

F. Worked Example

Consider two candidate moves evaluated on identical resulting positions: Move A yields (orbDelta, cellControl, corner, threat, eliminated, W) = (3, 6, 1, 1, 0, 2); Move B yields (5, 5, 0, 3, 0, 4). Under the Medium weights, Eq. (4) gives score(A) = 9.9 and score(B) = 10.4 — B is preferred, since its material and chain-length advantage outweighs its cost in exposed corner territory and threat. Under the Hard weights, the identical two positions score 12.7 and 12.3 respectively — the ranking *inverts*, because Hard's threat weight (1.4, versus Medium's 0.6) penalizes Move B's three exposed cells more heavily than the corresponding gain in material and chain length can compensate for. This single example demonstrates concretely that difficulty produces qualitatively different play (not merely noisier play) purely as a function of Table II, with no other code difference between tiers.

V. Implementation

A. Rendering and Interaction

The frontend renders the board on an HTML5 Canvas 2D context [4] at up to 60 Hz via `requestAnimationFrame`. Each orb is drawn as a small multi-layer radial gradient composite (an outer glow, a lit core sphere, a specular highlight, and a rim light) to suggest a three-dimensional glass sphere without any texture assets; cells approaching critical mass are given a sinusoidal position jitter and a pulsing radial warning aura, scaled by proximity to capacity, to communicate imminent danger ahead of an explosion. A lightweight particle system emits directional tracers and a radial burst on every explosion event the engine reports. All of this is driven purely by the engine's public event stream (Section III-A); the renderer holds no game logic of its own.

B. Procedural Audio

All sound is synthesized at runtime via the Web Audio API [5] — short envelope-shaped oscillator tones for placement, explosion, menu interaction, and a four-note victory arpeggio, plus a very low-level ambient tone while the setup menu is open — routed through a single shared gain node so that a single mute toggle silences everything, including in-flight sounds, without per-sound bookkeeping. This choice keeps the project's stated zero-runtime-dependency property intact for media as well as code: no audio file of any kind ships with or is fetched by the application.

C. Engineering for Robustness

Two robustness properties were added specifically in response to failure modes discovered during testing (Section VI-A) rather than anticipated up front, which we highlight because both are general lessons for this class of system rather than idiosyncrasies of this codebase. First, the cascade resolver enforces a hard ceiling on consecutive explosion waves (Section III-B); this is a direct consequence of the fact that chain-firing systems of this kind are not guaranteed by construction to terminate quickly on arbitrary or adversarial board configurations, so an explicit bound is required for a liveness guarantee, not merely a performance optimization. Second, every place in the frontend that hands off to an asynchronous bot "thinking" delay captures a monotonic session identifier at the start of that delay and re-validates it after the delay elapses before touching any shared game state; this was added after we found

that leaving a match (or restarting one) while a bot was mid-delay could otherwise let a stale callback dereference a null game reference or, more subtly, inject a move into whatever match had since replaced it, because the game object a restart operates on is reset in place rather than replaced, which defeats a naive object-identity staleness check.

D. Player-Setup UI: A Usability Case Study

The control that lets a user configure each player slot as human-controlled or as a bot at a chosen difficulty went through two materially different designs, and we record the transition here because the reason for it generalizes beyond this project. The first design packed each slot into a small circular token: a single click cycled the slot through Human → Easy → Medium → Hard → Human, and a small "×" remove affordance was overlaid on the token, made visible only on `:hover` and marked non-interactive (`pointer-events: none`) at every other time. This compiled and rendered without error and was not caught by either test tier active at the time, but was unusable in two independent ways: the remove control could not be activated on any device, because a permanently non-interactive element cannot receive a click regardless of its computed visibility, and — on a touch device specifically — it could not even be discovered, because `:hover` has no equivalent in a touch interaction model. Selecting a specific difficulty also required committing to a fixed number of clicks through an implicit, undocumented cycle order rather than a single direct action.

The redesign replaces the single overloaded control with three independent, always-visible controls per slot: an explicit two-way Human/Bot toggle, a separately visible Easy/Medium/Hard selector shown only once Bot is selected, and a remove control sized and styled for direct interaction with no hover dependency of any kind. Every state is reachable in exactly one action, and a slot's most recently chosen difficulty is remembered across a Human → Bot round-trip so switching a slot off and back on does not lose the user's prior choice. We report this not as a novel interaction pattern but as a concrete instance of a general principle — a control that depends on `:hover` for either its visibility or its interactivity does not degrade gracefully on touch input, it simply does not work — that is easy to violate unintentionally in exactly the way the first design did, and that neither a DOM-free unit test nor a casual desktop-mouse read-through of the code was positioned to catch.

E. Cross-Device and Responsive Support

The system targets both desktop and mobile browsers from a single layout rather than maintaining separate presentations. The canvas is sized in device-independent pixels and then scaled by `window.devicePixelRatio` at allocation time, so the board renders at native resolution on high-density mobile displays rather than being upscaled and appearing blurred; grid dimensions are recomputed and the canvas is reallocated on every resize and orientation change. Every interactive surface binds both a mouse `click` handler and a `touchend` handler that maps the touch point through the same coordinate-to-cell transform, rather than relying on a browser's synthetic mouse-event emulation for touch input, which is inconsistent enough across mobile browsers to be unreliable for precise grid-cell targeting. The visual theme itself depends on two modern CSS features — `backdrop-filter` for the glassmorphic panel styling and `:has()` for a state-reactive highlight on the active player's setup row — both broadly supported in current Chrome, Edge, Firefox, and Safari but with no fallback path for substantially older browsers, a deliberate scope decision given the project's zero-dependency, no-build-step constraint (Section V-B) rather than an oversight.

VI. Experimental Evaluation

A. Correctness and Regression Testing

The project is verified by 126 automated assertions across two tiers, summarized in Table III. The unit tier (98 assertions, plain Node.js, no installable dependency) exercises the engine and AI directly: construction and configuration validation, critical-mass computation, single and chained explosions, ownership transfer, turn rotation, the first-round elimination guard, the cascade safety-cap termination guarantee, and AI move legality and win-taking behavior. The end-to-end tier (28 assertions, a real headless-Chromium session driven via Playwright against the actual page) exercises the system the way a user would: menu configuration, full games played to completion, restart and rematch flows, and the operative (player-slot) management UI.

We report as a specific empirical finding that this second tier was not redundant with the first: three real defects were found only by driving the actual rendered, interactive page, and were invisible to both static code review and the DOM-free unit tier by construction. (i) A game configured with a computer-controlled player in the very first turn slot froze permanently with an empty board, because the frontend's only hook for triggering a bot's move was the engine's `turn_change` event, which — correctly, by the engine's own contract — is never emitted for the very first turn, since nothing has "changed" into it. (ii) A UI control (a button to leave the current match) became completely unclickable at every screen size once an unrelated feature (a persistent mute toggle) was added, because the two controls' CSS placement happened to overlap in the same screen corner and the newer element's stacking order silently won. (iii) A race condition, described in Section V-C, that could throw an uncaught exception or corrupt a second match's state depending on precisely when a user acted relative to an in-flight asynchronous delay. None of (i)–(iii) involve incorrect game logic in the sense a unit test targets; all three are properties of real layout, real event timing, or real user-triggered concurrency, and we regard this as a general argument for pairing headless-logic unit testing with real-browser end-to-end testing on any comparably interactive client-side project, not only this one.

Table III
Automated Test Coverage

Suite	Assertions	Environment
Engine unit tests	88	Node.js (no install)
AI unit tests	10	Node.js (no install)
Browser end-to-end tests	28	Headless Chromium (Playwright)
Total	**126**	0 failing at time of writing

B. AI Strength Across Difficulty Tiers

To evaluate whether the three difficulty tiers actually produce meaningfully different playing strength — as opposed to merely different labels on the same behavior — we ran a self-play tournament directly against the shipped engine and AI modules (not a re-implementation), pitting each pair of difficulty levels against each other over 100 games per pairing on a fixed 7×6 board, alternating which side moved first each game to cancel first-move advantage. Table IV and Fig. 2 report the results.

Table IV
Self-Play Tournament Results (100 games/pairing, 7×6 board)

Pairing	Win % (lower)	Win % (higher)	Avg. moves/game	Avg. largest cascade†
Easy vs. Medium	26.0	74.0	71.7	36.3
Easy vs. Hard	19.0	81.0	72.0	34.5
Medium vs. Hard	37.0	63.0	73.5	41.3
Hard vs. Hard‡ (baseline)	53.0	47.0	72.9	44.3

†Largest single move's cascade, measured in total exploding cells across every wave it triggers (not wave count). ‡Both contestants share a difficulty; columns report the first-named vs. second-named contestant, not a lower/higher tier.

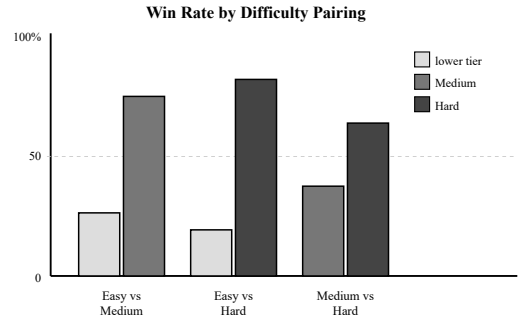

Fig. 2. Win rate of each difficulty pairing (Table IV, first three rows). The gradient is monotonic and consistent with the intended Easy < Medium < Hard ordering.

The win-rate gradient is monotonic and consistent with the intended ordering at every pairing: Hard defeats Easy in 81% of games and Medium in 63%; Medium defeats Easy in 74%. The narrower margin between Medium and Hard than between Easy and either is expected from Table II — Medium and Hard share every feature and differ only in degree, whereas Easy uniquely zeroes out the threat weight and substitutes a wide random pool three moves in four, which is a qualitative rather than incremental handicap. As a sanity check that this gradient reflects the evaluation function and not merely simulation noise, we additionally ran a Hard-vs-Hard self-play baseline (Table IV, final row), which produced a 53.0%/47.0% split — close to the 50/50 expected of two identically-weighted agents, and clearly distinct from the 74–81% margins observed at every cross-difficulty pairing — confirming the observed gradient is attributable to the weight differences in Table II and not an artifact of the tournament methodology itself. This baseline was not a formality: an earlier version of our measurement script mis-tracked wins by difficulty label rather than by which physical contestant actually won, which is invisible whenever the two labels differ but produced an impossible 100%/0% result the moment both contestants shared a label, which is exactly how the bug was caught (Section VII-B).

C. Decision Latency

Because move selection must complete well within the frontend's 600 ms artificial "thinking" delay to feel responsive, we measured wall-clock time for `getBestMove` under Node.js [8] across representative board sizes and all three difficulties, 50 samples each, on a scattered (non-empty) mid-game-like position. Table V reports the mean, 95th-percentile, and maximum observed latency for each configuration.

Table V
getBestMove Decision Latency (ms, 50 samples/cell)

Board	Difficulty	Mean	p95	Max
5×5	Easy	0.029	0.047	0.052
5×5	Medium	0.023	0.034	0.207
5×5	Hard	0.019	0.024	0.032
9×6	Easy	0.076	0.080	0.219
9×6	Medium	0.074	0.080	0.194
9×6	Hard	0.085	0.177	0.306
12×12	Easy	0.468	0.589	0.612
12×12	Medium	0.475	0.648	0.677
12×12	Hard	0.511	0.745	0.880

Even at the worst observed sample — 0.880 ms, on the largest supported 12×12 board — decision latency remains roughly three orders of magnitude under the 600 ms budget. Latency scales with board area (roughly 25× more cells between the 5×5 and 12×12 configurations corresponds to a roughly 20–25× increase in mean latency), as expected since every legal move on the board is individually simulated (Section IV-B), but shows no meaningfully different scaling behavior *across* difficulty tiers at a fixed board size: all three run the identical simulation-and-scoring procedure and differ only in the arithmetic applied to already-computed features, not in how many positions are simulated.

VII. Discussion

A. Interpreting the AI Results

The single-ply, simulation-grounded design (Section IV) is, we believe, well matched to this specific game's structure: because one move can recolor much of the board, evaluating the actual resulting position is more informative per unit of computation than searching shallowly over several moves evaluated more crudely. Its principal limitation is exactly what that framing implies — it has no model of an opponent's reply, so it cannot reason about multi-move plans or deliberately set traps that only pay off two turns later. A sufficiently deep search-based opponent, of the kind used in the comparable system discussed in Section II [2], could in principle exploit this and defeat the Hard tier consistently. We did not evaluate this directly — doing so would require implementing a comparable search-based opponent against a board of this branching factor, which we leave to future work (Section VIII) — but we view it as the most promising concrete extension of this project precisely because it targets the one capability the current design structurally lacks.

The worked example in Section IV-F and the tournament results in Section VI-B tell complementary parts of the same story: the former shows *why* the tiers should differ (a heavier threat weight inverts a concrete ranking), and the latter shows that this difference actually manifests as measurably different playing strength in practice rather than remaining a property visible only on paper. We consider this pairing of a symbolic argument with an empirical one more convincing than either alone would be — the worked example could in principle describe a difference too small to matter in practice, and the tournament result alone would not explain *why* the gradient exists or whether it was designed or accidental.

B. Threats to Validity

We identify three specific threats to the validity of the claims in Section VI, following the standard distinction between internal, external, and construct validity.

Internal validity. The tournament (Section VI-B) alternates which contestant moves first each game specifically to control for first-move advantage as a confound; we additionally verified the measurement methodology itself with a same-difficulty self-play baseline (Table IV, final row), confirming the win-tracking logic correctly attributes wins to the deciding board state rather than to an artifact of how contestants are labeled — an issue we in fact discovered and corrected during this evaluation, when an initial implementation mis-tracked wins by difficulty label rather than by which physical contestant won, an error that a same-label self-play baseline made immediately visible precisely because it should produce a near-50% split and initially did not.

External validity. All tournament results (Section VI-B) come from self-play among the system's own AI tiers on one fixed board size; we do not claim the specific win-rate magnitudes in Table IV generalize to human opponents, whose error patterns differ systematically from an evaluator optimizing the same six features the AI itself uses, or to substantially different board dimensions. We consider the qualitative, monotonic *ordering* more robust than the exact percentages, since it would be surprising for a weight vector that more heavily penalizes measurable threat and more heavily rewards measurable material to perform worse in expectation on a different board size, but we did not test this directly across sizes.

Construct validity. The six evaluation features (Table I) are a specific, hand-chosen operationalization of "board strength," not a canonical one; the AI's own worldview and our evaluation of it therefore share the same lens. A tournament between this AI and a differently featured opponent (Section VIII) would be a stronger test of whether these particular features

capture genuine strategic value or merely value relative to themselves.

C. Engineering Lessons

Table VI summarizes the three defects discussed in Section VI-A with their root cause and resolution. We highlight two engineering lessons here as, in our view, the most broadly transferable findings of this project, independent of the specific game: unbounded chip-firing/cascade systems (Section III-B) require an explicit termination bound as a *correctness* property, not only a performance optimization, since nothing in the rules themselves guarantees fast convergence on adversarial or highly symmetric inputs [9]; and any client-side application that hands off to a delayed asynchronous operation against shared mutable state needs an explicit staleness check keyed to something other than object identity whenever that state can be reset in place rather than replaced, since identity alone is exactly the check that a same-object reset silently defeats. Both were found only through interactive, real-browser testing (Section VI-A), which we take as reinforcing evidence for maintaining an end-to-end test tier on any project with comparable characteristics.

Table VI
Defects Found Only by End-to-End Testing (Section VI-A)

Defect	Root Cause	Resolution
Game frozen with player 0 as bot	`turn_change` never fires for the first turn	explicit kickoff call after match start/restart
"Leave match" button unclickable	CSS stacking overlap with an unrelated control	re-docked the overlapping control into the existing layout
Race condition on early exit	stale async callback acts on reset/null state	session-id guard re-checked after every await

VIII. Conclusion and Future Work

We have presented the design, implementation, and evaluation of *Chain Reaction: Nexus Protocol*, a dependency-free browser implementation of a chain-reaction capture-strategy game with a tiered, weighted-heuristic AI opponent. We formalized the cascade-resolution rule and the AI's move-simulation-and-scoring procedure, demonstrated with a worked example that difficulty is governed entirely by a weight vector rather than by separate logic paths, and supported these claims with 126 passing automated assertions across two independent test tiers and a 300-game empirical tournament showing the intended difficulty ordering holds. We further reported, as a general methodological finding, that three genuine defects were discoverable only through real-browser end-to-end testing and not through code review or DOM-free unit testing alone.

Directions for future work include: a bounded multi-ply search (e.g., shallow expectimax) layered on top of the existing single-ply evaluator, and a direct head-to-head comparison against a minimax-based opponent such as [2]; a networked or asynchronous multiplayer mode, since the current system is same-device pass-and-play only; persisted match statistics and a post-game summary; and broader accessibility, in particular a keyboard-operable path to place an orb, since board interaction is currently pointer/touch-only.

Acknowledgment

Portions of this project's code, tests, and documentation — including this report — were produced with the assistance of Anthropic's Claude (Claude Sonnet 5), an AI coding assistant, used interactively under the authors' direction and review throughout development; see reference [7] and the "Use of AI Tools" note accompanying this submission.

References

[1] Brilliant.org, "Chain Reaction (Game)," Brilliant Math & Science Wiki. [Online]. Available: https://brilliant.org/wiki/chain-reaction-game/

[2] A. D. Das and M. H. H. Papon, "chain-reaction-game," GitHub repository. [Online]. Available: https://github.com/MrArgho/chain-reaction-game

[3] C. E. Shannon, "Programming a Computer for Playing Chess," *Philosophical Magazine*, ser. 7, vol. 41, no. 314, 1950.

[4] Mozilla Developer Network, "Canvas API," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

[5] Mozilla Developer Network, "Web Audio API," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API

[6] Microsoft, "Playwright: Reliable end-to-end testing for modern web apps." [Online]. Available: https://playwright.dev/

[7] Anthropic, "Claude Sonnet 5" [Large language model], used as an AI coding assistant for implementation, testing, and documentation. [Online]. Available: https://www.anthropic.com/claude

[8] OpenJS Foundation, "Node.js," JavaScript runtime documentation. [Online]. Available: https://nodejs.org/

[9] C. J. Klivans, *The Mathematics of Chip-Firing*. Boca Raton, FL: CRC Press, 2019.